เอกสารประกอบการปฏิบัติการ
รายวิชา CP423324 Database and Web Security
Lab 8: Database Access Control and SQL Injection Protection

วัตถุประสงค์

1. เพื่อให้นักศึกษาสามารถกำหนดสิทธิ์ฐานข้อมูลตามหลัก Least Privilege ได้
2. เพื่อให้นักศึกษาสามารถทดสอบและเปรียบเทียบบัญชีที่มีสิทธิ์จำกัดกับบัญชีที่มีสิทธิ์มากเกินความจำเป็นได้
3. เพื่อให้นักศึกษาสามารถทดลองและอธิบาย SQL Injection ในระบบจำลองที่กำหนดได้
4. เพื่อให้นักศึกษาสามารถปรับปรุงโค้ดด้วย Prepared Statement ได้

คำชี้แจง

1. ให้นักศึกษา Capture ผลลัพธ์ตามที่งานของแต่ละข้อกำหนดลงใน Word และบันทึกไฟล์เป็น PDF โดยให้ตั้งชื่อว่า Lab08_รหัสนักศึกษา(มีขีด).pdf
2. เมื่อทำเสร็จเรียบร้อยแล้วให้แจ้งผู้ช่วยสอน (TA) เพื่อขอรับการตรวจและตอบคำถาม (กรุณาตรวจกับพี่ TA โดยคะแนนจะมาจากการตรวจในห้องเรียน)
3. เมื่อตรวจกับ TA เรียบร้อยแล้วให้นักศึกษาส่งใน Classroom เพื่อเป็นการสำรองข้อมูล
4. ไม่อนุญาตให้ส่งงานย้อนหลังได้ ยกเว้นกรณีลาป่วย หรืออุบัติเหตุ (ต้องมีใบรับรองแพทย์)

☆☆☆☆☆☆☆☆☆

โจทย์ที่กำหนด

งานปฏิบัติการนี้ให้นักศึกษาพัฒนาเว็บแอปพลิเคชันสำหรับค้นหาข้อมูลหนังสือ
โดยศึกษาความเสี่ยงจากการเชื่อมต่อฐานข้อมูลด้วยบัญชีที่มีสิทธิ์มากเกินความจำเป็น และความเสี่ยงจากการนำข้อมูลของผู้ใช้ไปต่อเป็นคำสั่ง SQL โดยตรง จากนั้นให้นักศึกษาสร้าง Database User สำหรับ Web Application ตามหลัก Least Privilege กำหนดเฉพาะสิทธิ์ SELECT, INSERT, UPDATE และ DELETE ทดลอง SQL Injection ในระบบจำลอง และปรับปรุงโค้ดด้วย Prepared Statement เพื่อเปรียบเทียบผลก่อนและหลังการแก้ไข

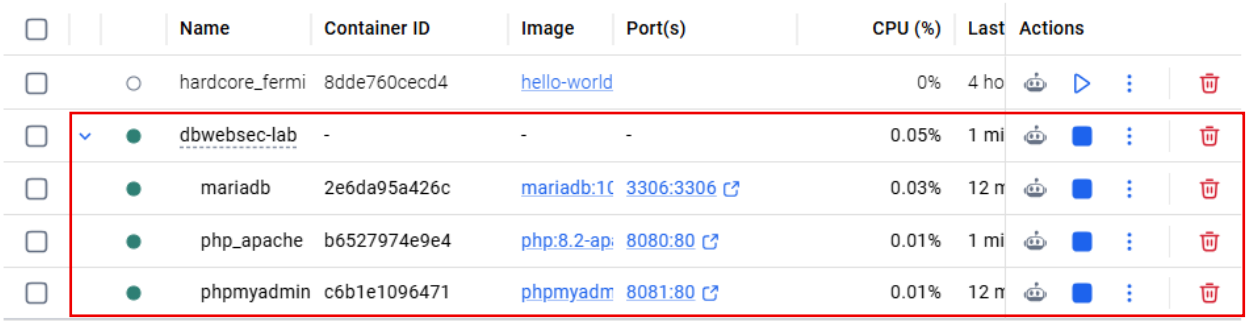
ลำดับการพัฒนาระบบ

1. เตรียมฐานข้อมูล db_lab08 และตาราง books

2. สร้างบัญชี webapp_user และกำหนดสิทธิ์ที่จำเป็น
3. ทดสอบสิทธิ์ของ webapp_user และเปรียบเทียบกับ root
4. สร้างไฟล์ Lab08_search_vulnerable.php และทดสอบการค้นหาปกติ
5. ทดลอง SQL Injection ในระบบจำลองและวิเคราะห์ SQL Query ที่เกิดขึ้น
6. สร้างไฟล์ Lab08_search_secure.php ซึ่งแก้ไขด้วย Prepared Statement และทดสอบ Input เดิมอีกครั้ง

โดยมีขั้นตอนการพัฒนาระบบ ดังนี้

1. เปิด docker ที่เคยสร้างไว้ ให้ Start container name: dbwebsec-lab, mariadb, php apache และphpmyadmin



2. การสร้างฐานข้อมูลและตารางผู้ใช้ ให้สร้างฐานข้อมูลชื่อ db_lab08 และตาราง books โดยมีโครงสร้างดังนี้

Field	Data Type	ข้อกำหนด	ความหมาย
id	INT	PRIMARY KEY, AUTO INCREMENT	รหัสหนังสือ
title	VARCHAR(150)	NOT NULL	ชื่อหนังสือ
author	VARCHAR(100)	NOT NULL	ชื่อผู้แต่ง
category	VARCHAR(50)	NOT NULL	หมวดหมู่
price	DECIMAL(8,2)	NOT NULL	ราคา

ตัวอย่างคำสั่ง SQL

สร้างฐานข้อมูล

```
CREATE DATABASE IF NOT EXISTS db_lab08
```

```
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

สร้างตาราง

```
    CREATE TABLE books (
      id INT AUTO_INCREMENT PRIMARY KEY,
      title VARCHAR(150) NOT NULL,
      author VARCHAR(100) NOT NULL,
      category VARCHAR(50) NOT NULL,
      price DECIMAL(8,2) NOT NULL
    );
```

จากนั้นเพิ่มข้อมูลตัวอย่าง

```
      INSERT INTO books (title, author, category, price) VALUES
      ('Database Fundamentals', 'John Smith', 'Database', 450.00),
      ('Learning PHP', 'Anna Lee', 'Programming', 390.00),
      ('Web Security Basics', 'David Kim', 'Security', 520.00),
      ('SQL for Beginners', 'Mary Jones', 'Database', 350.00),
      ('Secure Web Development', 'Peter Brown', 'Security',
      590.00);
```

โดยเมื่อเพิ่มข้อมูลแล้วจะมีข้อมูลในตาราง books ดังนี้

id	title	author	category	price
1	Database Fundamentals	John Smith	Database	450.00
2	Learning PHP	Anna Lee	Programming	390.00
3	Web Security Basics	David Kim	Security	520.00
4	SQL for Beginners	Mary Jones	Database	350.00
5	Secure Web Development	Peter Brown	Security	590.00

3. สร้าง Database User สำหรับ Web Application

สำหรับระบบเว็บแอปพลิเคชันไม่ควรเชื่อมต่อฐานข้อมูลด้วยบัญชี root ในการทำงานปกติ เนื่องจาก root มีสิทธิ์มากกว่าที่ Web Application จำเป็นต้องใช้ จึงให้สร้างบัญชี webapp_user สำหรับใช้งานเฉพาะกับฐานข้อมูล db_lab08

การสร้าง User มีคำสั่ง ดังนี้

```
CREATE USER IF NOT EXISTS
'webapp_user'@'%'
IDENTIFIED BY 'WebApp1234';
```

การกำหนดสิทธิ์ในการใช้งานฐานข้อมูลให้ webapp_user โดยให้สามารถ SELECT, INSERT, UPDATE, DELETE ในทุกตารางของฐานข้อมูล db_lab08 ได้ มีคำสั่ง ดังนี้

```
GRANT SELECT, INSERT, UPDATE, DELETE
ON db_lab08.*
TO 'webapp_user'@'%';
```

หมายเหตุ: หลัก Least Privilege

ผู้ใช้ควรได้รับเฉพาะสิทธิ์ที่จำเป็นต่อหน้าที่เท่านั้น สำหรับ Web Application นี้ต้องอ่าน เพิ่ม แก้ไข และลบข้อมูลหนังสือ จึงต้องการเพียง SELECT, INSERT, UPDATE และ DELETE ไม่จำเป็นต้องมี CREATE, DROP, ALTER หรือ GRANT OPTION

งานที่ 1.1 ตรวจสอบ Database User และ Privileges
1) เปิด phpMyAdmin และรัน SHOW GRANTS FOR 'webapp_user'@'%';
2) Capture หน้าจอผลลัพธ์

งานที่ 1.2 ทดสอบหลัก Least Privilege
1) ให้เปิด CMD แล้ว Login ด้วย ผู้ใช้ webapp_user ด้วยคำสั่ง

```
docker exec -it mariadb mariadb -u webapp_user -p
```

แล้วใส่รหัสผ่าน WebApp1234

```
C:\Users\Lenovo>docker exec -it mariadb mariadb -u webapp_user -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 263
Server version: 10.11.18-MariaDB-ubu2204 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

2) ใช้ webapp_user รัน SELECT * FROM db_lab08.books; และ Capture ผลลัพธ์

3) ทดลอง DROP TABLE db_lab08.books;และ Capture Error inที่เกิดขึ้น

4. การสร้างเว็บแอปพลิเคชัน

ให้นักศึกษาสร้างโฟลเดอร์ Lab08 ไว้ภายใน C:\dbwebsec-lab\www สำหรับไฟล์ที่ต้องสร้างมีไฟล์ทั้งหมด 3 ไฟล์ มีรายละเอียด ดังนี้

1) Lab08_config.php เชื่อมต่อฐานข้อมูลด้วย webapp_user
2) Lab08_search_vulnerable.php หน้าค้นหาที่สร้าง SQL ด้วยการต่อ String เพื่อใช้สาธิต SQL Injection
3) Lab08_search_secure.php หน้าค้นหาที่ใช้ Prepared Statement เพื่อป้องกัน SQL Injection (นักศึกษาสร้างเอง)

4.1 การสร้างไฟล์ Lab08_config.php โดยมีคำสั่ง ดังนี้

```php
<?php
declare(strict_types=1);

date_default_timezone_set('Asia/Bangkok');

const DB_HOST = 'db';
const DB_PORT = 3306;
const DB_NAME = 'db_lab08';
const DB_USER = 'webapp_user';
const DB_PASS = 'WebApp1234';

mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

try {
    $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME, DB_PORT);
    $conn->set_charset('utf8mb4');
    $conn->query("SET time_zone = '+07:00'");
} catch (mysqli_sql_exception $e) {
    http_response_code(500);
    error_log($e->getMessage());
    exit('Database connection failed.');
}

function h(?string $value): string
{
    return htmlspecialchars($value ?? '', ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

สังเกต: ไฟล์นี้ใช้ webapp_user แทน root เพื่อจำกัดสิทธิ์ของบัญชีที่ Web Application ใช้เชื่อมต่อฐานข้อมูล

4.2 สร้างไฟล์ Lab08_search_vulnerable.php ไฟล์นี้ใช้เพื่อสาธิตความเสี่ยงจากการนำ User Input ไปต่อเป็น SQL String โดยตรง ให้ทดลองเฉพาะในระบบ Lab นี้เท่านั้น

```php
<?php
declare(strict_types=1);
require_once 'Lab08_config.php';

$keyword = $_GET['keyword'] ?? '';
$rows = [];
$queryError = '';

$sql = "SELECT id, title, author, category, price
        FROM books
        WHERE title LIKE '%$keyword%'";

if ($keyword !== '') {
    try {
        $result = $conn->query($sql);
        while ($row = $result->fetch_assoc()) {
            $rows[] = $row;
        }
    } catch (mysqli_sql_exception $e) {
        error_log($e->getMessage());
        $queryError = 'Query failed.';
    }
}
?>
```

```php
<!doctype html>
<html lang="th">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Lab08 - Vulnerable Search</title>

<style>
body{font-family:Arial,sans-serif;background:#f3f6f9;margin:0}
.container{width:min(950px,94%);margin:40px auto;
    background:#fff;padding:28px;border-radius:12px;
    box-shadow:0 8px 30px rgba(0,0,0,.08)}
input{width:75%;padding:10px;box-sizing:border-box}
button{padding:10px 18px;border:0;border-radius:8px;
    background:#dc3545;color:#fff;cursor:pointer}
table{width:100%;border-collapse:collapse;margin-top:22px}
th,td{border:1px solid #ddd;padding:10px;text-align:left}
th{background:#f1f3f5}.warning{padding:12px;background:#fff3cd;margin-bottom:18px}
.sql{margin-top:18px;padding:12px;background:#eef2f7;white-space:pre-wrap;
    word-break:break-word}.error{margin-top:18px;padding:12px;
    background:#f8d7da;color:#842029}
</style>
</head>

<body><div class="container">
<h1>Book Search - Vulnerable Version</h1>
<div class="warning">ใช้เพื่อสาธิต SQL Injection ในระบบจำลองของ Lab08 เท่านั้น</div>
<form method="get">
<input type="text" name="keyword" value="<?= h($keyword) ?>" placeholder="Search book title">
<button type="submit">Search</button>
</form>
<div class="sql"><strong>SQL generated by application:</strong><br><?= h($sql) ?></div>
<?php if ($queryError !== ''): ?><div class="error"><?= h($queryError) ?></div><?php endif; ?>
<?php if ($keyword !== '' && $queryError === ''): ?>
<table><thead><tr>
    <th>ID</th><th>Title</th>
    <th>Author</th><th>Category</th>
    <th>Price</th>
    </tr></thead><tbody>
    <?php if (!$rows): ?><tr><td colspan="5">No data found.</td></tr><?php else: ?>
    <?php foreach ($rows as $row): ?><tr>
        <td><?= (int)$row['id'] ?></td>
        <td><?= h($row['title']) ?></td>
        <td><?= h($row['author']) ?></td>
        <td><?= h($row['category']) ?></td>
        <td><?= number_format((float)$row['price'],2) ?></td>
    </tr><?php endforeach; ?>
    <?php endif; ?>
</tbody></table>
<?php endif; ?>
<p><a href="Lab08_search_secure.php">ไปยัง Secure Version</a></p>
</div></body></html>
```

งานที่ 2.1 ทดสอบ Vulnerable Version ด้วยข้อมูลปกติ

1) เปิด Browser และกรอก URL http://localhost:8080/Lab08/Lab08_search_vulnerable.php Capture หน้าจอผลลัพธ์

2) กรอกคำค้น Database แล้วกด Search
3) Capture คำสั่งจาก Generated SQL generated by application และ Capture หน้าจอผลลัพธ์ที่แสดงข้อมูลหนังสือที่มีคำว่า Database

งานที่ 2.2 ทดลอง SQL Injection ในระบบจำลอง
1) กรอก Input: ' OR '1'='1' -- แล้วกด Search
2) Capture คำสั่งจาก Generated SQL generated by application และ Capture หน้าจอผลลัพธ์ที่แสดงข้อมูลหนังสือ

4.3 ให้สร้างไฟล์ Lab08_search_secure.php ด้วยตัวเอง โดยแก้ไขคำสั่งจากไฟล์ Lab08_search_vulnerable.php เพื่อป้องกัน SQL Injection ได้

งานที่ 3.1 ทดสอบ Secure Version
1) เปิด http://localhost:8080/Lab08/Lab08 search_secure.php
2) ทดลอง Input เดียวกับงานที่ 2.2 คือ ' OR '1'='1' --
3) Capture ผลลัพธ์

งานที่ 3.2 Capture คำสั่งให้ไฟล์ Lab08_search_secure.php